Задача 1. Простая сортировка

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В первой строке записано целое число N — количество записей ($1 \le N \le 10\ 000$). В остальных N строках содержатся записи, по одной в строке.
Для каждой записи указаны ключ и значение через пробел. Ключ — это целое число в диапазоне от 0 до 10^9 включительно, а значение — это строка от одного до семи символов включительно, состоящая только из маленьких букв латинского алфавита.
Требуется вывести ровно те же самые N записей, но в другом порядке. Записи должны быть упорядочены по возрастанию ключа. Если у нескольких записей ключ равный, то нужно упорядочить их в том порядке, в котором они встречаются по входном файле.

Пример

input.txt	output.txt
7 3 qwerty 3 string 6 good 1 a 3 ab 2 hello 5 world	1 a 2 hello 3 qwerty 3 string 3 ab 5 world 6 good

Пояснение к примеру

В примере 7 записей с ключами 1, 2, 3, 5 и 6 — именно в таком порядке записи и выведены в выходном файле. Обратите внимание, что есть три записи с ключом 3: qwerty, string, ab. Они выведены ровно в том порядке, в котором они идут во входном файле.

Задача 2. Сортировка слиянием

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В первых четырёх байтах входного файла задано целое число N — количество чисел в массиве A. Далее идут N четырёхбайтовых целых чисел — содержимое массива A. Размер массива лежит в диапазоне: $0 \le N \le 500\,000$.

Требуется отсортировать массив A по не убыванию, используя алгоритм сортировки слиянием.

В выходной файл нужно вывести ровно N четырёхбайтовых целых чисел: содержимое массива A после сортировки.

Пример

input.txt	output.txt
0A 00 00 00 1F 00 00 00 F2 FF FF FF 06 00 00 00 04 00 00 00 26 00 00 00 FD FF FF FF 1E 00 00 00 F6 FF FF FF 0A 00 00 00 F4 FF FF FF	F2 FF FF FF F4 FF FF FF F6 FF FF FF FD FF FF FF 04 00 00 00 06 00 00 00 0A 00 00 00 1E 00 00 00 1F 00 00 00 26 00 00 00

Задача 3. Удаление дубликатов

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дан массив A, в котором содержится n целых чисел. Нужно удалить из него дубликаты (т.е. повторы чисел), так чтобы в массиве каждое имеющееся в нём значение встречалось ровно один раз.

Формат входного файла

В первой строке записано целое число n — сколько чисел в массиве ($1 \le N \le 300\ 000$). В остальных n строках записаны сами эти числа. Все числа по модулю не превышают 10^9.)

Формат выходного файла

В первой строке нужно вывести целое число k — сколько различных чисел в массиве A. В оставшихся k строках нужно вывести сами эти различные числа в любом порядке.

Пример

input.txt	output.txt
10 1 1 -2 4 3 0 0 0 0 -2	5 1 3 -2 0 4

Комментарий

Подсказка: подумайте, как решить задачу за $O(N)$, если массив A упорядочен по возрастанию.

Задача 4. Сумма минимумов

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дан массив A, в котором содержится n целых чисел. Нужно перебрать все пары чисел A_i и A_j в этом массиве, для каждой пары найти минимум $\min(A_i, A_j)$ и сложить вместе все эти минимумы.
Более формально, требуется найти сумму:

$$S = \sum_{i<k} \min(A_i, A_j) = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} \min(A_i, A_j)$$

Формат входного файла

В первой строке записано целое число n — сколько чисел в массиве ($1 \le N \le 300\,000$).

В остальных n строках записаны самы эти числа в том порядке, в котором они идут в массиве A. Все числа по модулю не превышают 10^9.)

Формат выходного файла

Нужно вывести одно целое число — искомую сумму минимумов S.

Осторожно: сумма S может быть довольно большой. Оцените максимально возможное значение S и выберите подходящий целочисленный тип.

Пример

input.txt	output.txt
5 1 4 3 5 6	26

Комментарий

Подсказка: подумайте, как решить задачу за $O(N)$, если массив A упорядочен по возрастанию.

Задача 5. Быстрая сортировка

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В первых четырёх байтах входного файла задано целое число N — количество чисел в массиве A. Далее идут N четырёхбайтовых целых чисел — содержимое массива A. Размер массива лежит в диапазоне: $0 \le N \le 500\,000$.
Требуется отсортировать массив A по не убыванию, используя алгоритм быстрой сортировки.
В выходной файл нужно вывести ровно N четырёхбайтовых целых чисел: содержимое массива A после сортировки.

Пример

input.txt
0A 00 00 00 1F 00 00 00 F2 FF FF FF 06 00 00 00 04 00 00 00 26 00 00 00 FD FF FF FF 1E 00 00 00 F6 FF FF FF 0A 00 00 00 F4 FF FF FF
output.txt
F2 FF FF FF F4 FF FF FF F6 FF FF FF FD FF FF FF 04 00 00 00 06 00 00 00 0A 00 00 00 1E 00 00 00 1F 00 00 00 26 00 00 00

Задача 6. Поразрядная сортировка

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В первых четырёх байтах входного файла задано целое число N — количество элементов в массиве A. Далее записано N элементов, каждый элемент состоит из ключа и значения.
Ключ и значение являются беззнаковыми 32-битными целыми числами. Размер массива лежит в диапазоне: $0 \le N \le 20\,000\,000$ (внимание: $2 \cdot 10^7$).
Требуется отсортировать массив A по неубыванию ключа, **используя алгоритм поразрядной сортировки** (radix sort). Чтобы решение работало быстро, рекомендуется разбивать 32-битный ключ на четыре цифры, по одному байту каждая цифра.
В выходной файл нужно вывести N элементов: содержимое массива A после сортировки. Так же как и во входных данных, каждый элемент должен состоять из ключа и значения.

Пример

input.txt
0A 00 00 00 1F 00 00 00 1E 00 00 00 0E 00 00 00 24 00 00 00 05 00 00 00 1B 00 00 00 10 00 00 00 06 00 00 00 22 00 00 00 11 00 00 00 0A 00 00 00 11 00 00 00 25 00 00 00 05 00 00 00 0E 00 00 00 1D 00 00 00 25 00 00 00 1D 00 00 00 0E 00 00 00 1B 00 00 00
output.txt
05 00 00 00 1B 00 00 00 0A 00 00 00 11 00 00 00 0E 00 00 00 24 00 00 00 0E 00 00 00 1D 00 00 00 0E 00 00 00 1B 00 00 00 10 00 00 00 06 00 00 00 1F 00 00 00 1E 00 00 00 22 00 00 00 11 00 00 00 25 00 00 00 05 00 00 00 25 00 00 00 1D 00 00 00

Задача 7. Сортировка кучей

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В первых четырёх байтах входного файла задано целое число N — количество чисел в массиве A. Далее идут N четырёхбайтовых целых чисел — содержимое массива A. Размер массива лежит в диапазоне: $0 \le N \le 500\,000$.

Требуется отсортировать массив A по не убыванию, используя алгоритм сортировки кучей. В простейшем случае алгоритм состоит в том, чтобы добавить все элементы массива в двоичную кучу, а затем последовательно извлекать из кучи минимальный/максимальный элемент и записывать обратно в массив.

В выходной файл нужно вывести ровно N четырёхбайтовых целых чисел: содержимое массива A после сортировки.”.

Пример

input.txt
0A 00 00 00 1F 00 00 00 F2 FF FF FF 06 00 00 00 04 00 00 00 26 00 00 00 FD FF FF FF 1E 00 00 00 F6 FF FF FF 0A 00 00 00 F4 FF FF FF
output.txt
F2 FF FF FF F4 FF FF FF F6 FF FF FF FD FF FF FF 04 00 00 00 06 00 00 00 0A 00 00 00 1E 00 00 00 1F 00 00 00 26 00 00 00

Задача 8. Адаптивная сортировка массива

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	2 секунды
Ограничение по памяти:	разумное

Вам требуется реализовать адаптивный алгоритм сортировки массива, который эффективно работает на различных входных данных. Алгоритм должен:

1. Для массивов размером $n \leq k$ (где k — порог, например, 50) использовать квадратичную сортировку вставками (Insertion Sort).
2. Для массивов размером $n > k$ начинать с быстрой сортировки (QuickSort) с выбором опорного элемента как медианы из трех (первый, средний, последний элементы).
3. Во время выполнения QuickSort отслеживать h - глубину рекурсии. Если глубина превышает h > $2 \cdot \log_2(n)$, переключиться на сортировку кучей (HeapSort), которая гарантирует сложность $O(n\log\ n)$
4. Для подмассивов размером $\leq k$ в QuickSort также использовать Insertion Sort.

Время k и h вы можете подобрать, запуская массивы с разным k со следующими командами:

gcc main.c -o main

time ./main

Уметь объяснить для чего происходит такое разделение.

Формат входного файла

В первой строке входного файла содержится одно целое число n — размер массива ($1 \leq n \leq 10^6$). Во второй строке задано n целых чисел a_i ($-10^9 \leq a_i \leq 10^9$) — элементы массива.

Формат выходного файла

Выведите n чисел — элементы массива в отсортированном порядке. Числа в строке разделяются пробелами.

Пример

input.txt	output.txt
9 5 2 8 1 9 3 6 4 7	1 2 3 4 5 6 7 8 9

Задача 9. Булевы формулы

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	2 секунды
Ограничение по памяти:	разумное

Задано две булевы формулы, первая является полиномом Жегалкина, а вторая представлена в конъюнктивной нормальной форме. Требуется определить, эквивалентны они или нет.

Формат входного файла

В первой строке записано целое число T — количество тестов в файле ($1 \le T \le 100$).
Далее описывается T тестов, по две строки на тест. В первой строке теста записан полином Жегалкина, а во второй — формула в КНФ.
Каждая формула записывается без пробелов и состоит только из маленьких латинских букв от a до j (первые 10 букв) и следующих символов:
1. '&' (ASCII 38) — конъюнкция,
2. '|' (ASCII 124) — дизъюнкция (только в КНФ),
3. '!' (ASCII 33) — отрицание (только в КНФ),
4. '+' (ASCII 43) — сложение по модулю два (только в полиноме Жегалкина),
5. '1' (ASCII 49) — истина (только в полиноме Жегалкина).
Буквы обозначают различные булевы переменные.
Обе формулы синтаксически корректны с точки зрения алгебры логики. В формулах отсутствуют скобки, так что в КНФ надо считать, что дизъюнкция имеет больший приоритет, чем конъюнкция. В КНФ перед каждой переменной либо стоит один символ отрицания, либо отрицания нет (гарантируется, что кратных символов отрицания нет). Гарантируется, что константа истины 1 не умножается ни на какую переменную, а встречается в сумме только сама по себе.
Длина каждой формулы не превышает 5 000 символов.

Формат выходного файла

Требуется вывести T строк, в каждой строке ответ на соответствующий тест. Если формулы эквивалентны, нужно написать Equivalent, а иначе — Not equivalent.

Пример

input.txt	output.txt
a&b&c+1 !a|b&a|!b&c a+b+c a|b|c&!a|!b|c&!a|b|!c&a|!b|!c	Not equivalent Equivalent

Пояснение к примеру

В первом тесте заданы неэквивалентные формулы: $(a \wedge b \wedge c) \oplus 1$ и $(\bar{a} \vee b) \wedge (a \vee \bar{b}) \wedge c$.

Во втором тесте заданы эквивалентные формулы: $a \oplus b \oplus c$ и $(a \vee b \vee c) \wedge (\bar{a} \vee \bar{b} \vee c) \wedge (\bar{a} \vee b \vee \bar{c}) \wedge (a \vee \bar{b} \vee \bar{c})$.

Задача 10. Маленькая сортирующая машина

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Вам предлагается испытать себя в оптимизации сортировки массива маленького размера.
В каждом тесте имеется массив из N элементов, у каждого элемента есть ключ и значение. И ключ, и значение являются беззнаковыми четырёхбайтовыми целыми числами.
Изначально во входном файле заданы только значения всех элементов массива. Далее нужно выполнить R раундов. На каждом раунде нужно:
1. Сгенерировать и записать N случайных чисел в ключи элементов массива.
2. Отсортировать элементы массива в порядке возрастания ключа.
В результате каждого раунда значения в массиве переставляются в некотором порядке, который зависит от генератора псевдослучайных чисел.
В данной задаче нужно использовать генератор псевдослучайных чисел xorwow. Исходный код этого генератора:

```
uint32_t xorwow(uint32_t state[5]) {
    /* Algorithm "xorwow" from p. 5 of Marsaglia, "Xorshift RNGs" */
    uint32_t s, t = state[3];
    t ^= t >> 2;
    t ^= t << 1;
    state[3] = state[2]; state[2] = state[1]; state[1] = s = state[0];
    t ^= s;
    t ^= s << 4;
    state[0] = t;
    return t + (state[4] += 362437);
}
```

Начальное состояние генератора (пять чисел в state) задаётся в каждом тесте. В начале каждого раунда ключи генерируются для элементов в их текущем порядке, причём старшие два бита отбрасываются:

```
for (int i = 0; i < n; i++)
    elements[i].key = xorwow(state) & 0 x3fffffff;
```

В конце теста нужно вывести значения всех элементов массива в их финальном порядке. Обратите внимание, что процесс полностью детерминирован, и только один финальный порядок является правильным.

Формат входного файла

В первой строке записано целое число Q — сколько тестов записано в файле ($1 \le Q \le 1\,000$).
Далее описано Q тестов.

Описание теста начинается со строки с двумя целыми числами: N — размер массива и R — сколько раундов сортировки нужно выполнить ($1 \le N \le 64$, $0 \le R$). Во второй строке теста записано пять шестнадцатеричных чисел, по восемь цифр в каждом — начальное содержимое массива state генератора xorwow. В третьей строке записано N целых чисел — значения элементов массива в том порядке, в котором они расположены изначально.

Гарантируется, что во всех раундах всех тестов все ключи будут различными. Суммарное количество раундов R по всем тестам в файле не превышает 1 500 000.

Формат выходного файла

Для каждого из Q тестов нужно вывести ровно одну строку. В этой строке должно быть N целых чисел: значения элементов массива в их финальном порядке после всех раундов.

Пример

input.txt
5 15 0 b1c6114b f18c80b8 059cace1 24e9297b 5cab5281 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 15 1 b1c6114b f18c80b8 059cace1 24e9297b 5cab5281 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 15 2 b1c6114b f18c80b8 059cace1 24e9297b 5cab5281 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 15 3 b1c6114b f18c80b8 059cace1 24e9297b 5cab5281 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 10 7 4c373cdb 0102026b a8b5ef27 370796de 5840f014 135 12 13 11 10 17 10 7 1 5
output.txt
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 2 6 5 10 3 12 8 14 13 11 4 9 7 15 1 1 11 12 4 5 3 6 14 2 10 8 15 9 13 7 5 12 1 11 8 13 14 15 3 10 2 9 7 6 4 12 1 11 13 135 10 7 10 17 5

Комментарий

В данной задаче бессовестно жёсткое ограничение по времени. Во входном файле под номером $5 + R$ записано 10 тестов с $50\,000 \cdot R$ раундами суммарно и с максимальным N (для каждого $R = 1 \ldots 30$).

Внимание: не пытайтесь применять многопоточность! В боте замеряется суммарное процессорное время по всем потокам, поэтому многопоточность не поможет

Задача 11. Большая сортирующая машина

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В каждом тесте имеется массив из N элементов, у каждого элемента есть ключ и значение. Ключ является 64-битным беззнаковым целым числом, а значение — 32-битным.
Изначально каждому элементу массива присваивается значение, равное его номеру (считая с нуля). Далее нужно выполнить R раундов. На каждом раунде нужно:
1. Сгенерировать и записать N случайных чисел в ключи элементов массива.
2. Отсортировать элементы массива в порядке возрастания ключа.
В результате каждого раунда значения в массиве переставляются в некотором порядке, который зависит от генератора псевдослучайных чисел. В данной задаче нужно использовать генератор псевдослучайных чисел xorshift+. Исходный код этого генератора:

```
uint64_t xorshift128plus(uint64_t state[2]) {
    uint64_t x = state[0];
    uint64_t const y = state[1];
    state[0] = y;
    x ^= x << 23; // a
    state[1] = x ^ y ^ (x >> 17) ^ (y >> 26); // b, c
    return state[1] + y;
}
```

Начальное состояние генератора (два числа в state) задаётся в каждом тесте. В начале каждого раунда ключи генерируются для элементов в их текущем порядке, причём старшие два бита отбрасываются:

```
for (int i = 0; i < n; i++)
    elements[i].key = xorshift128plus(state) & 0 x3fffffffffffffffLL;
```

Обратите внимание, что процесс полностью детерминирован, значит порядок элементов после выполнения всех раундов определяется однозначно. В конце теста нужно вывести 64-битный хеш от финального порядка элементов:

```
uint64_t hash = 5381;
for (int i = 0; i < n; i++)
    hash = hash * 31 + elements[i].value;
```

Формат входного файла

В первой строке записано целое число Q — сколько тестов записано в файле ($1 \le Q \le 10$). Далее описано Q тестов.

Описание теста начинается со строки с двумя целыми числами: N — размер массива и R — сколько раундов сортировки нужно выполнить ($1 \le N \le 10^6$, $0 \le R$). Во второй строке теста записано два шестнадцатеричных числа, по шестнадцать цифр в каждом — начальное содержимое массива state генератора xorshift+.

Гарантируется, что во всех раундах всех тестов все ключи будут различными. Суммарное количестов раундов R по всем тестам в файле не превышает 40. Кроме того, сумма всех N в файле не превышает 10^6.

Формат выходного файла

Для каждого из Q тестов нужно вывести одно целое число в отдельной строке — финальный хеш. Хеш следует выводить в виде шестнадцатеричного числа из ровно 16 цифр (если нужно, добавьте ведущие нули).

Пример

input.txt
5 15 0 b1c6114bf18c80b8 059cace124e9297b 15 1 b1c6114bf18c80b8 059cace124e9297b 15 2 b1c6114bf18c80b8 059cace124e9297b 15 3 b1c6114bf18c80b8 059cace124e9297b 10 7 4c373cdb0102026b a8b5ef27370796de
output.txt
25d1acb755e9cd42 f9237b1061947488 f1af6c757ccad3ba 7238a85181dfa6bc 3d356e76dc3e5d86

Комментарий

Финальный порядок элементов (elements[i].value) в пяти тестах примера такой:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
8 14 10 6 0 7 4 12 2 11 1 3 13 9 5
2 12 6 1 0 9 4 8 5 3 10 7 14 13 11
11 4 1 0 6 14 9 3 12 8 10 2 7 5 13
5 1 0 4 2 7 9 3 6 8

В данной задаче бессовестно жёсткое ограничение по времени. Во входном файле под номером $5 + R$ записано 10 тестов с $50\,000 \cdot R$ раундами суммарно и с максимальным N (для каждого $R = 1 \ldots 30$).

Внимание: не пытайтесь применять многопоточность! В боте замеряется суммарное процессорное время по всем потокам, поэтому многопоточность не поможет